

02477 Bayesian Machine Learning 2025: Assignment 2

This is the second assignment out of three in the Bayesian machine learning course 2026. The assignment is a group work of 2-4 students (please use the same groups as in assignment 1 if possible) and hand in via DTU Learn). The assignment is part of the curriculum, but handing in the assignment is **optional**. Handing in your solution is an opportunity to get feedback on your learning. The deadline for handing in is **9th of April 23:59**. The solution must be handed in as a single **PDF** document.

Part 1: Gaussian processes and covariance functions

In this part, we will study covariance functions for Gaussian process models. Consider the following six covariance functions

$$k_1(x, x') = 2 \exp \left(-\frac{(x - x')^2}{2 \cdot 0.3^2} \right) \quad (1)$$

$$k_2(x, x') = \exp \left(-\frac{(x - x')^2}{2 \cdot 0.1^2} \right) \quad (2)$$

$$k_3(x, x') = 4 + 2xx' \quad (3)$$

$$k_4(x, x') = \exp \left(-2 \sin(3\pi \cdot |x - x'|)^2 \right) \quad (4)$$

$$k_5(x, x') = \exp \left(-2 \sin(3\pi \cdot |x - x'|)^2 \right) + 4xx' \quad (5)$$

$$k_6(x, x') = \frac{1}{5} + \min(x, x') \quad (6)$$

Some of them should be familiar and some of them might be new to you.

Task 1.1: Determine the analytical marginal prior mean and variance of a Gaussian process, $f_i(x) \sim \mathcal{GP}(0, k_i(x, x'))$, i.e. compute $\mathbb{E}[f_i(x)]$ and $\mathbb{V}[f_i(x)]$ for each of the six covariance functions (for $i = 1, 2, \dots, 6$).

Solution

The prior mean for each process is 0 by construction. The marginal prior variance of a Gaussian process is given by $\mathbb{V}[f_i(x)] = k_i(x, x)$. Therefore, we have

$$\mathbb{V}[f_1(x)] = 2 \exp \left(-\frac{0^2}{2 \cdot 0.3^2} \right) = 2 \quad (7)$$

$$\mathbb{V}[f_2(x)] = \exp \left(-\frac{0^2}{2 \cdot 0.1^2} \right) = 1 \quad (8)$$

$$\mathbb{V}[f_3(x)] = 4 + 2xx = 4 + 2x^2 \quad (9)$$

$$\mathbb{V}[f_4(x)] = \exp \left(-2 \sin(3\pi \cdot |0|)^2 \right) = 1 \quad (10)$$

$$\mathbb{V}[f_5(x)] = \exp \left(-2 \sin(3\pi \cdot |0|)^2 \right) + 4x^2 = 1 + 4x^2 \quad (11)$$

$$\mathbb{V}[f_6(x)] = \frac{1}{5} + \min(x, x) = \frac{1}{5} + x \quad (12)$$

End of solution

Task 1.2: Which of the six covariance functions are stationary covariance functions?

Solution

Covariance functions k_1 , k_2 , and k_4 are stationary, whereas k_3 , k_5 and k_6 are not. For example, if we let $\tau = x - x'$, then we can rewrite k_1 as a function of τ , i.e.

$$k_1(x, x') = k_1(\tau) = 2 \exp\left(-\frac{\tau^2}{2 \cdot 0.3^2}\right), \quad (13)$$

which is not possible for the non-stationary functions.

End of solution

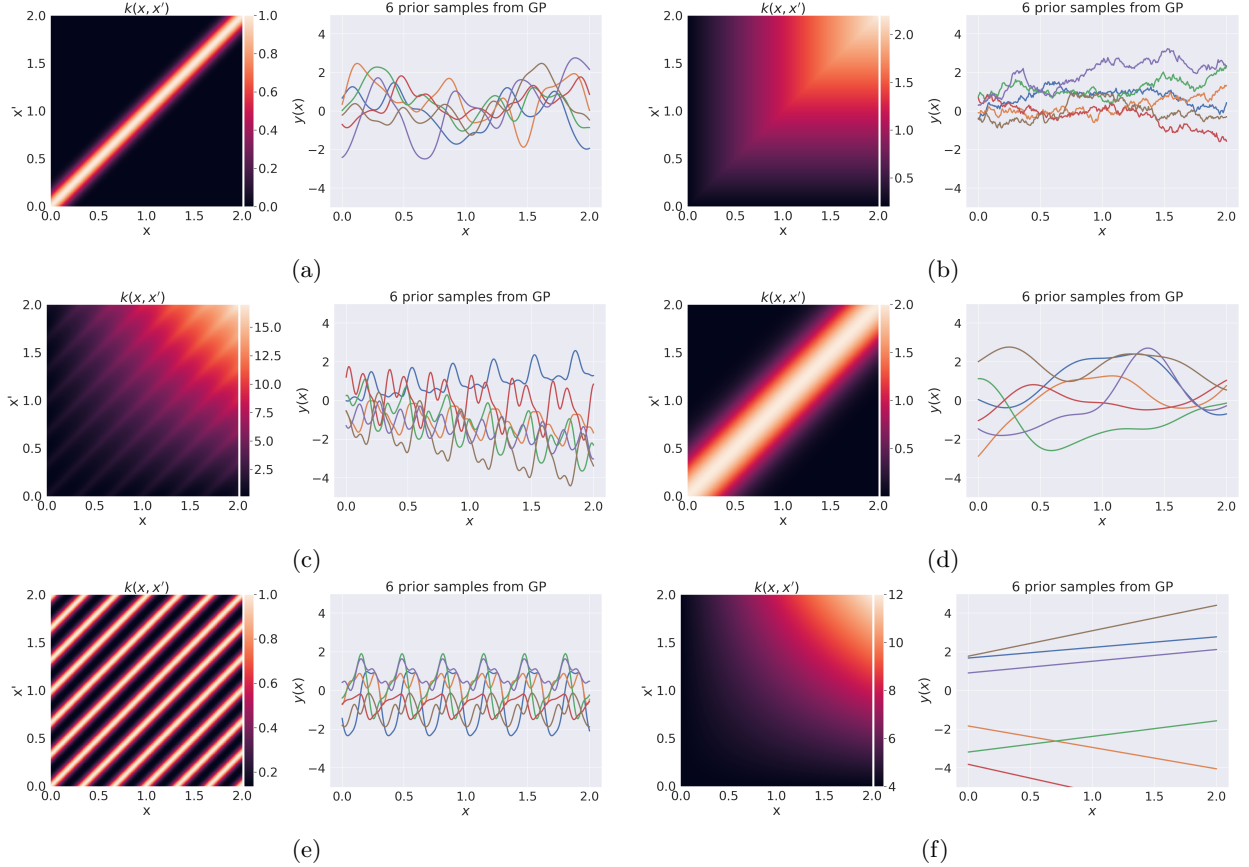


Figure 1: Covariance matrices and samples from the corresponding Gaussian process prior distribution for the six different covariance functions in randomized order.

Task 1.3: Let $X = \{x_i\}_{i=1}^{100}$ be a sorted set of equidistant points in the interval $[0, 2]$. **Figure 1** shows the covariance matrices for function values evaluated at X as well as samples from the corresponding Gaussian process prior for each of the six covariance functions. Match the plots to each of the six covariance functions.

Solution

First, we recognize k_1 and k_2 as squared exponential (exponentiated quadratic) kernels with a (relative) long and short lengthscale, respectively. Therefore, we have that k_1 correspond to (d) and k_2 corresponds to (a) as evidence both from the structure of the covariance matrix as well as the samples functions. Next, we recognize k_3 as the sum of a constant kernel and a linear kernel, which matches panel (f). As a sanity check, we concluded earlier that $\mathbb{V}[y_3(x)] = 4 + 2x^2$, and therefore $\mathbb{V}[y_3(0)] = 4$, which is also consistent with panel (f). Observing that kernels k_4 and k_5 both contain a periodic component, we conclude that they

must correspond to either (c) or (e). Noting that k_5 also contain a linear component, we can deduce that k_5 matches (c) and k_4 matches (e). Finally, this means that k_6 must correspond to (b).

The matching pairs are summarized below:

Covariance function	Plot
1	d
2	a
3	f
4	e
5	c
6	b

End of solution

Consider now the following kernel function:

$$k_7(x, x') = \kappa_0^2 + \kappa_1^2 x x' + \kappa_2^2 \exp\left(-\frac{\|x - x'\|_2^2}{2\ell^2}\right), \quad (14)$$

where $\kappa_1, \kappa_2 \geq 0$ and $\ell > 0$ are hyperparameters.

Task 1.4: Implement the kernel function k_7 . Generate and plot $S = 30$ realizations (samples) of the process $f(x) \sim \mathcal{GP}(0, k_7(x, x'))$ for $x \in [-6, 6]$ for $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 0, \frac{1}{2})$. Repeat with $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 0, 1, \frac{1}{2})$ and $(\kappa_0, \kappa_1, \kappa_2, \ell) = (5, 2, 1, \frac{1}{2})$

Hint: You can either adapt the code template from the GP exercises or you can simply implement the relevant equations directly.

Solution

See figure 2.

End of solution



Figure 2: Prior samples for Task 1.4

Part 2: Laplace approximation for a simple neural network

Consider the following small regression data set with $N = 10$ observations $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$, where

$$\mathbf{x} = [9.589, 7.375, 4.647, 2.501, 2.538, 6.783, 4.294, 5.111, 0.130, 0.783] \quad (15)$$

$$\mathbf{y} = [3.032, 3.349, 2.906, 2.126, 1.538, 2.787, 3.078, 2.993, 0.828, -0.331] \quad (16)$$

such that the n 'th element in $\mathbf{x}$ is $x_n \in \mathbb{R}$ and similar for $y_n \in \mathbb{R}$.

We will work with the following very simple two-parameter neural network:

$$f(x) = w_1 \sigma(x + w_0), \quad (17)$$

where $\mathbf{w} = [w_0 \ w_1] \in \mathbb{R}^2$ is the parameters of the network and $\sigma : \mathbb{R} \rightarrow (0, 1)$ is the logistic sigmoid function. We can construct a non-linear probabilistic model for regression by using the network $f(x)$ as the

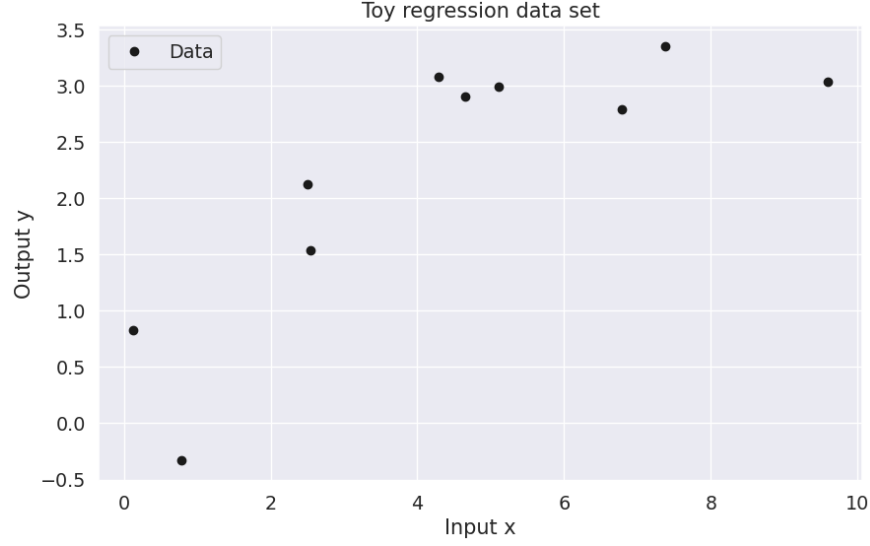


Figure 3: Data for Part 2

mean function of a Gaussian likelihood:

$$p(y_n|x_n, \mathbf{w}) = \mathcal{N}(y_n|f(x), \beta^{-1}) = \mathcal{N}(y_n|w_1\sigma(x + w_0), \beta^{-1}), \quad (18)$$

where $\beta^{-1} > 0$ is the noise variance. We impose i.i.d. Gaussian prior distributions on both parameters:

$$w_0 \sim \mathcal{N}(0, \tau^2) \quad (19)$$

$$w_1 \sim \mathcal{N}(0, \tau^2), \quad (20)$$

where $\tau^2 > 0$ is variance of the prior.

Assume $\tau = 2$ and $\beta = 4$.

Task 2.1: Generate $S = 100$ samples from the prior $p(\mathbf{w})$ and plot the corresponding functions $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.

Hint: You can find inspiration in the course notebooks in order to make the plots look nice.

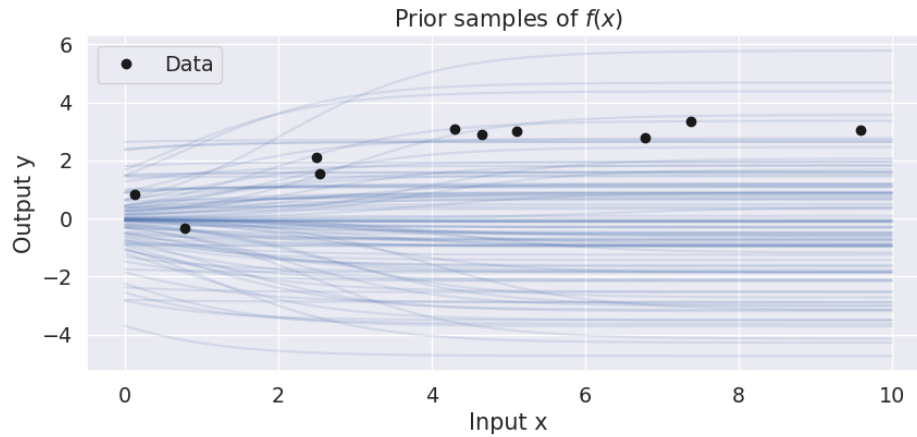
Solution

See Figure 4

End of solution

The joint density for $(\mathbf{y}, \mathbf{w})$ according to the probabilistic model in eq. (18)-(20) is given

$$p(\mathbf{y}, \mathbf{w}) = p(\mathbf{w}) \prod_{n=1}^N p(y_n|x_n, \mathbf{w}) = \mathcal{N}(w_0|0, \tau^2) \mathcal{N}(w_1|0, \tau^2) \prod_{n=1}^N \mathcal{N}(y_n|w_1\sigma(x_n + w_0), \beta^{-1}) \quad (21)$$



```

1 # hyperparameters
2 beta = 4
3 tau = 2.
4
5 # implement sigmoid and neural network f(x)
6 sigmoid = lambda x: 1./(1 + jnp.exp(-x))
7 f = lambda x, w: w[1]*sigmoid(x + w[0])
8
9 # number of samples, random seed etc.
10 S = 100
11 key = random.PRNGKey(1)
12 xpred = jnp.linspace(0, 10, 1000)[: , None]
13
14 # generate prior samples of the model
15 w_prior = random.multivariate_normal(key, jnp.zeros(2), tau**2*jnp.identity(2), shape=(S))
16
17 # make plot
18 fig, ax = plt.subplots(1, 1, figsize=(10, 4))
19 for wi in w_prior:
20     ax.plot(xpred, f(xpred, wi), 'b-', alpha=0.15)
21 ax.plot(xtrain, ytrain, 'ko', label='Data')
22 ax.set(xlabel='Input x', ylabel='Output y', title='Prior samples of $f(x)$')
23 ax.legend()
24

```

Figure 4: Solution for task 2.1

Task 2.2: What prevents us from using the equations in section 3.3 in Murphy1 to compute the exact posterior distribution of the parameters given the data analytically for the system in eq. (21) ?

Solution

Even though both prior and likelihood are Gaussians, we cannot use the equations from section 3.3 in Murphy1 because of the *non-linear* dependency on w_0 in the mean of the likelihood.

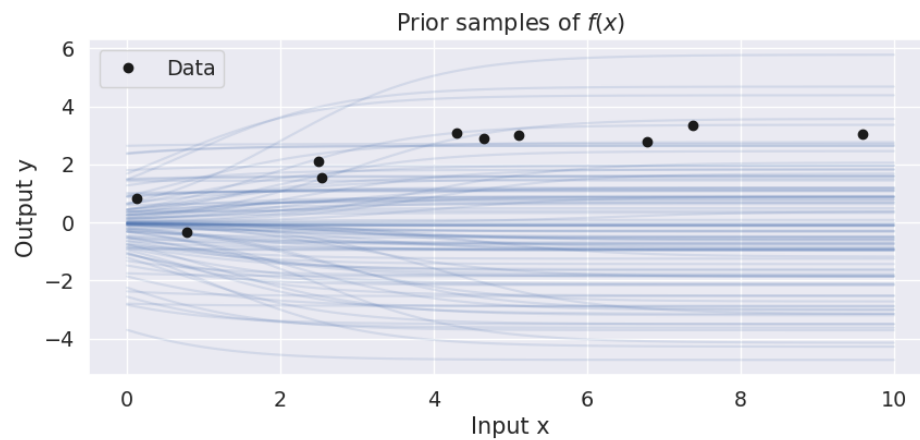
End of solution

Task 2.3: Implement a python for function for evaluating the *logarithm of the joint density* in eq. (21). Report the numerical value of $\log p(y, w)$ for the dataset given above when $w_0 = w_1 = 0$.

Solution

Evaluating the logarithm of the joint density at the zero vector yields $\log p(y, \mathbf{0}) = -130.949$. See Figure 5.

End of solution



```

1 def log_lik(w):
2     ftrain = f(xtrain, w)
3     log_lik_y = log_npdpf(ytrain, ftrain, 1./beta)
4     return log_lik_y.sum(0)[0]
5
6 def log_prior(w):
7     return log_npdpf(w[0], 0, tau**2) + log_npdpf(w[1], 0, tau**2)
8
9 def log_joint(w):
10    return log_prior(w) + log_lik(w)
11
12 print(f'Evaluating the log joint at w = [0, 0]: {log_joint(jnp.zeros(2)):4.3f}')
13

```

Figure 5: Solution for task 2.3

Task 2.4: Determine the expression for the gradient and Hessian of the logarithm of the joint density wrt. w .

Hints: 1) The chain rule for differentiation will be handy. 2) Recall the identities $\frac{d}{dx}\sigma(x) = \sigma'(x) = \sigma(x)(1 - \sigma(x))$ and $\frac{d^2}{dx^2}\sigma(x) = \sigma''(x) = (1 - 2\sigma(x))\sigma'(x)$. 3) Start by determining the gradient and then validate your result via the next two tasks and then come and work on the Hessian.

Solution

First, we write out the expression for the log joint density and get rid of terms that are constant wrt. w_0 and/or w_1 :

$$\log p(\mathbf{y}, \mathbf{w}) = \log \mathcal{N}(w_0|0, \tau^2) + \log \mathcal{N}(w_1|0, \tau^2) + \sum_{n=1}^N \log \mathcal{N}(y_n|w_1\sigma(x_n + w_0), \beta^{-1}) \quad (22)$$

$$= -\frac{1}{2} \log(2\pi\tau^2) - \frac{1}{2\tau^2}w_0^2 - \frac{1}{2} \log(2\pi\tau^2) - \frac{1}{2\tau^2}w_1^2 + \sum_{n=1}^N \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{\beta}{2}(y_n - w_1\sigma(x_n + w_0))^2 \right] \quad (23)$$

$$= -\frac{1}{2\tau^2}w_0^2 - \frac{1}{2\tau^2}w_1^2 - \frac{\beta}{2} \sum_{n=1}^N (y_n - w_1\sigma(x_n + w_0))^2 + \text{constant} \quad (24)$$

Next, we compute the derivative wrt. w_0

$$\frac{\partial}{\partial w_0} \log p(\mathbf{y}, \mathbf{w}) = -\frac{1}{2\tau^2} \frac{\partial}{\partial w_0} w_0^2 - \frac{1}{2\tau^2} \frac{\partial}{\partial w_0} w_1^2 - \frac{\beta}{2} \sum_{n=1}^N \frac{\partial}{\partial w_0} (y_n - w_1\sigma(x_n + w_0))^2 + \frac{\partial}{\partial w_0} \text{constant} \quad (25)$$

$$= -\frac{1}{\tau^2}w_0 - \frac{\beta}{2} \sum_{n=1}^N \frac{\partial}{\partial w_0} (y_n - w_1\sigma(x_n + w_0))^2 \quad (26)$$

$$= -\frac{1}{\tau^2}w_0 - \frac{\beta}{2} \sum_{n=1}^N 2(y_n - w_1\sigma(x_n + w_0)) \left[\frac{\partial}{\partial w_0} y_n - w_1 \frac{\partial}{\partial w_0} \sigma(x_n + w_0) \right] \quad (27)$$

$$= -\frac{1}{\tau^2}w_0 + \beta \sum_{n=1}^N (y_n - w_1\sigma(x_n + w_0)) w_1 \frac{\partial}{\partial w_0} \sigma(x_n + w_0) \quad (28)$$

$$= -\frac{1}{\tau^2}w_0 + \beta \sum_{n=1}^N (y_n - w_1\sigma(x_n + w_0)) w_1 \sigma(x_n + w_0) (1 - \sigma(x_n + w_0)) \quad (29)$$

$$= -\frac{1}{\tau^2}w_0 + \beta \sum_{n=1}^N (y_n - f(x_n)) w_1 \sigma'(x_n + w_0) \quad (30)$$

And the derivative wrt. w_1

$$\frac{\partial}{\partial w_1} \log p(\mathbf{y}, \mathbf{w}) = -\frac{1}{2\tau^2} \frac{\partial}{\partial w_1} w_0^2 - \frac{1}{2\tau^2} \frac{\partial}{\partial w_1} w_1^2 - \frac{\beta}{2} \sum_{n=1}^N \frac{\partial}{\partial w_1} (y_n - w_1 \sigma(x_n + w_0))^2 + \frac{\partial}{\partial w_1} \text{constant} \quad (31)$$

$$= -\frac{1}{\tau^2} w_1 - \frac{\beta}{2} \sum_{n=1}^N 2(y_n - w_1 \sigma(x_n + w_0)) \frac{\partial}{\partial w_1} [y_n - w_1 \sigma(x_n + w_0)] \quad (32)$$

$$= -\frac{1}{\tau^2} w_1 + \beta \sum_{n=1}^N (y_n - w_1 \sigma(x_n + w_0)) \sigma(x_n + w_0) \quad (33)$$

$$= -\frac{1}{\tau^2} w_1 + \beta \sum_{n=1}^N (y_n - f(x_n)) \sigma(x_n + w_0) \quad (34)$$

Hence, the gradient wrt. $\mathbf{w}$ becomes

$$\nabla_{\mathbf{w}} \log p(\mathbf{y}, \mathbf{w}) = -\frac{1}{\tau^2} \mathbf{w} + \beta \sum_{n=1}^N (y_n - f(x_n)) \begin{bmatrix} w_1 \sigma'(x_n + w_0) \\ \sigma(x_n + w_0) \end{bmatrix} \quad (35)$$

We can now compute the Hessian matrix $\mathcal{H} \in \mathbb{R}^{2 \times 2}$ element by element. First,

$$\mathcal{H}_{11} = \frac{\partial^2}{\partial w_0^2} \log p(\mathbf{y}, \mathbf{w}) \quad (36)$$

$$= -\frac{\partial}{\partial w_0} \frac{1}{\tau^2} w_0 + \beta \sum_{n=1}^N \frac{\partial}{\partial w_0} (y_n - f(x_n)) f(x_n) (1 - \sigma(x_n + w_0)) \quad (37)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N \left[\frac{\partial}{\partial w_0} (y_n f(x_n) (1 - \sigma(x_n + w_0)) - f(x_n) f(x_n) (1 - \sigma(x_n + w_0))) \right] \quad (38)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N \left[\frac{\partial}{\partial w_0} (y_n w_1 \sigma(x_n + w_0) (1 - \sigma(x_n + w_0)) - w_1^2 \sigma(x_n + w_0)^2 (1 - \sigma(x_n + w_0))) \right] \quad (39)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N \left[\frac{\partial}{\partial w_0} (y_n w_1 \sigma'(x_n + w_0) - w_1^2 \sigma(x_n + w_0) \sigma'(x_n + w_0)) \right] \quad (40)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N [(y_n w_1 \sigma''(x_n + w_0) - w_1^2 [\sigma'(x_n + w_0) \sigma'(x_n + w_0) + \sigma(x_n + w_0) \sigma''(x_n + w_0)])] \quad (41)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N [(y_n w_1 - w_1^2 \sigma(x_n + w_0)) \sigma''(x_n + w_0) - (w_1 \sigma'(x_n + w_0))^2] \quad (42)$$

And

$$\mathcal{H}_{22} = \frac{\partial^2}{\partial w_1^2} \log p(\mathbf{y}, \mathbf{w}) \quad (43)$$

$$= -\frac{1}{\tau^2} \frac{\partial}{\partial w_1} w_1 + \beta \sum_{n=1}^N \frac{\partial}{\partial w_1} (y_n - f(x_n)) \sigma(x_n + w_0) \quad (44)$$

$$= -\frac{1}{\tau^2} + \beta \sum_{n=1}^N \frac{\partial}{\partial w_1} (y_n \sigma(x_n + w_0) - f(x_n) \sigma(x_n + w_0)) \quad (45)$$

$$= -\frac{1}{\tau^2} - \beta \sum_{n=1}^N \frac{\partial}{\partial w_1} f(x_n) \sigma(x_n + w_0) \quad (46)$$

$$= -\frac{1}{\tau^2} - \beta \sum_{n=1}^N \frac{\partial}{\partial w_1} w_1 \sigma(x_n + w_0) \sigma(x_n + w_0) \quad (47)$$

$$= -\frac{1}{\tau^2} - \beta \sum_{n=1}^N \sigma(x_n + w_0)^2. \quad (48)$$

Finally, the cross term

$$\mathcal{H}_{12} = \frac{\partial}{\partial w_0} \frac{\partial}{\partial w_1} \log p(\mathbf{y}, \mathbf{w}) \quad (49)$$

$$= -\frac{1}{\tau^2} \frac{\partial}{\partial w_0} w_1 + \beta \sum_{n=1}^N \frac{\partial}{\partial w_0} (y_n - f(x_n)) \sigma(x_n + w_0) \quad (50)$$

$$= \beta \sum_{n=1}^N \frac{\partial}{\partial w_0} (y_n \sigma(x_n + w_0) - f(x_n) \sigma(x_n + w_0)) \quad (51)$$

$$= \beta \sum_{n=1}^N \left[y_n \frac{\partial}{\partial w_0} \sigma(x_n + w_0) - \frac{\partial}{\partial w_0} f(x_n) \sigma(x_n + w_0) \right] \quad (52)$$

$$= \beta \sum_{n=1}^N \left[y_n \sigma'(x_n + w_0) - \frac{\partial}{\partial w_0} w_1 \sigma(x_n + w_0) \sigma(x_n + w_0) \right] \quad (53)$$

$$= \beta \sum_{n=1}^N \left[y_n \sigma'(x_n + w_0) - \frac{\partial}{\partial w_0} w_1 \sigma(x_n + w_0)^2 \right] \quad (54)$$

$$= \beta \sum_{n=1}^N \left[y_n \sigma'(x_n + w_0) - 2w_1 \sigma(x_n + w_0) \frac{\partial}{\partial w_0} \sigma(x_n + w_0) \right] \quad (55)$$

$$= \beta \sum_{n=1}^N [y_n \sigma'(x_n + w_0) - 2w_1 \sigma(x_n + w_0) \sigma'(x_n + w_0)] \quad (56)$$

$$= \beta \sum_{n=1}^N (y_n - 2w_1 \sigma(x_n + w_0)) \sigma'(x_n + w_0) \quad (57)$$

End of solution

Task 2.5: Determine the Laplace approximation $q(w)$ of the posterior distribution such that $p(w|\mathbf{y}) \approx q(w)$.

Solution

The Laplace approximation for $p(\mathbf{w}|\mathbf{y})$ is given by $q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{m}, \mathbf{\Sigma})$, where $\mathbf{m} = \mathbf{w}_{\text{MAP}}$ and $\mathbf{\Sigma} = -\mathcal{H}^{-1}$ and $\mathcal{H}$ is the negative Hessian of the log joint evaluated at $\mathbf{w}_{\text{MAP}}$. Since we have the gradient, we can obtain $\mathbf{w}_{\text{MAP}}$ is a simple first-order optimization scheme:

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \alpha g(\mathbf{w}^t), \quad (58)$$

where $g(\mathbf{w}) = \nabla_{\mathbf{w}} \log p(\mathbf{y}, \mathbf{w})$ and t is the iteration index. Using $\alpha = 2 \cdot 10^{-2}$ and running 200 iterations starting from origo yields

$$\hat{\mathbf{w}}_{\text{MAP}} = \begin{bmatrix} -2.204 \\ 3.104 \end{bmatrix} \quad (59)$$

where the squared norm of the gradient is approximately 10^{-17} . Evaluating the Hessian at $\hat{\mathbf{w}}_{\text{MAP}}$ yields:

$$\mathcal{H} = \begin{bmatrix} -7.854 & -6.897 \\ -6.897 & -25.143 \end{bmatrix} \quad (60)$$

such that the resulting covariance matrix becomes

$$\mathbf{\Sigma} = -\mathcal{H}^{-1} \approx \begin{bmatrix} 0.168 & -0.046 \\ -0.046 & 0.052 \end{bmatrix}. \quad (61)$$

See Figure 6 for the code.

End of solution

```

1 # first and second order gradient of logistic sigmoid
2 grad_sigmoid = lambda x: sigmoid(x)*(1-sigmoid(x))
3 grad2_sigmoid = lambda x: (1-2*sigmoid(x))*sigmoid(x)*(1-sigmoid(x))
4
5 def gradient(w):
6     """ implements gradient of log joint """
7     # precompute
8     fx = model(xtrain, w)
9     sigmoid_x = sigmoid(xtrain+w[0])
10    grad_sigmoid_x = grad_sigmoid(xtrain+w[0])
11
12    # coordinate wise gradient
13    g0 = -w[0]/tau**2 + beta*jnp.sum((ytrain - fx)*w[1]*grad_sigmoid_x)
14    g1 = -w[1]/tau**2 + beta*jnp.sum((ytrain - fx)*sigmoid_x)
15    return jnp.array([g0, g1])
16
17
18 def hessian(w):
19     """ implements Hessian of log joint """
20     # precompute
21     fx = model(xtrain, w)
22     sigmoid_x = sigmoid(xtrain+w[0])
23     g1 = grad_sigmoid(xtrain+w[0])
24     g2 = grad2_sigmoid(xtrain+w[0])
25
26     # compute each entry of Hessian matrix
27     H11 = -1./tau**2 + beta*jnp.sum((ytrain*w[1] - w[1]**2*sigmoid_x)*grad2_sigmoid(xtrain+w
28     [0]) - (w[1]*g1)**2)
29     H22 = -1./tau**2 - beta*jnp.sum(sigmoid_x**2)
30     H12 = beta*jnp.sum((ytrain-2*w[1]*sigmoid_x)*g1)
31
32     return jnp.array([[H11, H12], [H12, H22]])
33
34 # optimization loop
35 w_MAP = jnp.array([0., 0.])
36 step_size = 2e-2
37 for itt in range(200):
38     # compute gradient
39     g = gradient(w_MAP)
40
41     # update
42     w_MAP = w_MAP + step_size*g
43
44     # report progress
45     if (itt+1) % 50 == 0:
46         print(f'Itt {itt:4d}: log joint = {log_joint(w_MAP):4.3f}, ||g||^2 = {jnp.sum(g**2)
47         :3.2e}')
48
49 print(f'\nCoordinates for w_MAP, i.e. mean for Laplace')
50 print(np.array2string(w_MAP, precision=3))
51
52 print(f'\n Hessian at w_MAP')
53 H = hessian(w_MAP)
54 print(np.array2string(H, precision=3))
55
56 Sigma = -jnp.linalg.inv(H)
57 print(f'\n Covariance matrix for Laplace approximation')
58 print(np.array2string(Sigma, precision=3))

```

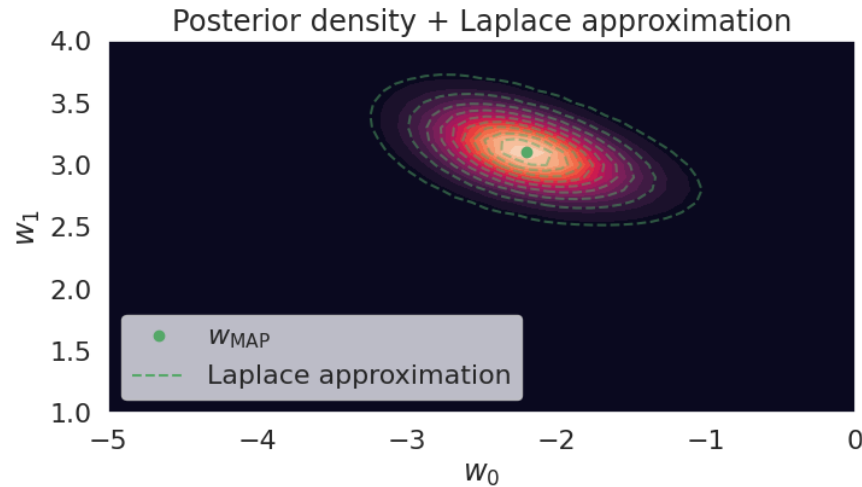
Figure 6: Solution for task 2.5

Task 2.6: Create a 2D contour plot of posterior distribution of the parameters. Plot the contours of the Laplace approximation on top of the plot to validate your results from the previous tasks.

Hints: Find inspiration in the exercise notebook for week 3 & 4 if you are struggling with the plotting code.

Solution

See Figure 7.



```

1 from scipy.stats import multivariate_normal
2
3 # define grid
4 w0_grid = jnp.linspace(-5, 5, 100)
5 w1_grid = jnp.linspace(-5, 5, 101)
6
7 # evaluate joint
8 Z = jnp.zeros((len(w0_grid), len(w1_grid)))
9 for i in range(len(w0_grid)):
10     for j in range(len(w1_grid)):
11         wi = jnp.array([w0_grid[i], w1_grid[j]])
12         Z = Z.at[i,j].set(log_joint(wi))
13
14 # evaluate approximation
15 q_logpdf = jnp.zeros((len(w0_grid), len(w1_grid)))
16 for i in range(len(w0_grid)):
17     for j in range(len(w1_grid)):
18         wi = jnp.array([w0_grid[i], w1_grid[j]])
19         q_logpdf = Z.at[i,j].set(multivariate_normal.logpdf(wi, w_MAP, Sigma))
20
21 # plot
22 fig, ax = plt.subplots(1, 1, figsize=(8, 4))
23 ax.contourf(w0_grid, w1_grid, jnp.exp(Z.T), 20)
24 ax.plot(w_MAP[0], w_MAP[1], 'go', label='$w_{\text{MAP}}$')
25 ax.set(xlabel='$w_0$', ylabel='$w_1$', title='Posterior density + Laplace approximation',
26        xlim=(-5, 0), ylim=(1, 4))
27 ax.contour(w0_grid, w1_grid, jnp.exp(q_logpdf.T), jnp.linspace(1e-6, jnp.exp(log_joint(w_MAP
28        )), 10), colors='g', linestyle='--', alpha=0.5)
29 ax.plot(0, 0, 'g--', label='Laplace approximation')
30 ax.legend(loc='lower left')

```

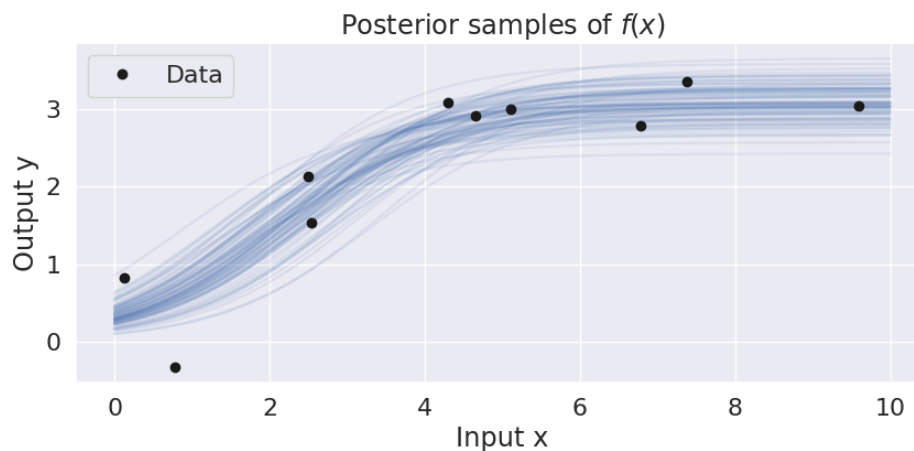
Figure 7: Contour plot of the posterior density and Laplace approximation

End of solution

Task 2.7: Generate and plot $S = 100$ samples from the approximate posterior distribution. Plot the corresponding function $f(x)$ for $x \in [0, 10]$ on top of a scatter plot of the data.

Solution

See Figure 8.



```

1 # generate samples
2 S = 100
3 w_posterior = random.multivariate_normal(key, w_MAP, Sigma, shape=(S))
4
5 # make plot
6 fig, ax = plt.subplots(1, 1, figsize=(10, 4))
7 for wi in w_posterior:
8     ax.plot(xpred, model(xpred, wi), 'b-', alpha=0.1)
9
10 ax.plot(xtrain, ytrain, 'ko', label='Data')
11 ax.set(xlabel='Input x', ylabel='Output y', title='Posterior samples of $f(x)$')
12

```

Figure 8: Posterior samples of $f(x)$

End of solution

Task 2.8: Use the Laplace approximation to compute the (approximate) posterior probability for the event $f(8) > 3$.

Solution

We can estimate the requested probability using a Monte Carlo estimator based on samples from the Laplace approximation as follows.

$$\begin{aligned}
 p(f(8) > 3 | \mathbf{y}) &= \int \mathbb{I}[f(8; \mathbf{w}) > 3] p(\mathbf{w} | \mathbf{y}) d\mathbf{w} \\
 &\approx \int \mathbb{I}[f(8; \mathbf{w}) > 3] q(\mathbf{w}) d\mathbf{w} && \text{(Laplace approx.)} \\
 &\approx \frac{1}{S} \sum_{i=1}^S \mathbb{I}[f(8; \mathbf{w}^{(i)}) > 3] && \text{(Monte Carlo approx.)} \\
 &\approx 0.660 (\pm 0.015)
 \end{aligned}$$

where $\mathbf{w}^{(i)} \sim q(\mathbf{w})$.

```

1 # generate samples
2 S = 1000
3 w_posterior = random.multivariate_normal(key, w_MAP, Sigma, shape=(S))
4
5 # compute f(8) for each posterior sample
6 fstar = jnp.array([model(8, wi) for wi in w_posterior])
7
8 # estimate probability and MCSE
9 event = fstar > 3
10 MCSE = jnp.sqrt(jnp.var(event)/S)
11 print(f'p(f(8)) > 3 = {jnp.mean(event):4.3f} (MCSE={MCSE:4.3f})')
12

```

Figure 9: Solution task 2.8

End of solution

Task 2.9: Determine the mean and a 95% interval for the (approximate) distribution $p(y^*|\mathbf{y}, x^*)$ for $x^* = 3.75$.

Solution

We can estimate the requested mean and interval using Monte Carlo estimation. First, we generate $S = 1000$ samples from $p(y^*|\mathbf{y}, x^* = 3.75)$ using Ancestral sampling and then we compute the statistics (see Fig. 10), yielding:

$$\mathbb{E}_{p(y^*|\mathbf{y}, x^*=3.75)}[y^*] \approx \frac{1}{S} \sum_{i=1}^S y_i^*, \quad (62)$$

where $y_{(i)}^* \sim p(y^*|\mathbf{y}, x^* = 3.75)$ is sampled via ancestral sampling:

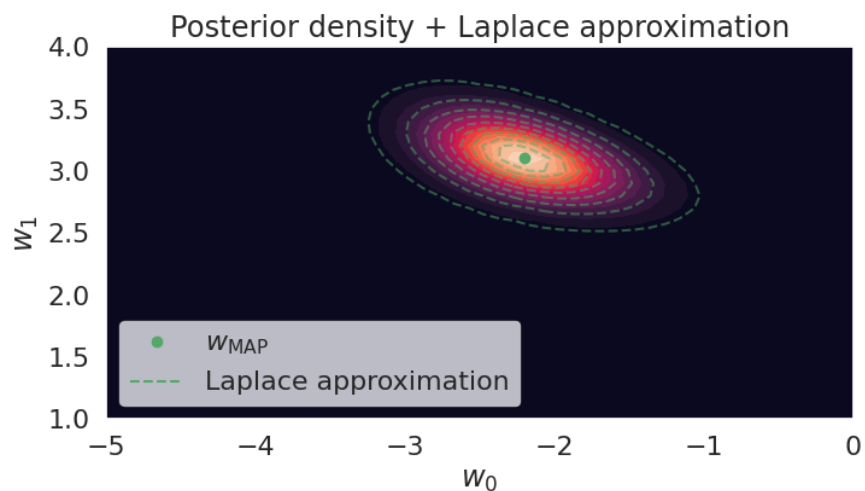
$$y_{(i)}^* = w_1^{(i)} \sigma(x^* + w_0^{(i)}) + \epsilon^{(i)} \quad (63)$$

for $\epsilon_{(i)} \sim \mathcal{N}(0, \beta^{-1})$ and $\mathbf{w}^{(i)} = \begin{bmatrix} w_0^{(i)} & w_1^{(i)} \end{bmatrix}$ is the i 'th posterior sample.

Similarly, we can estimate a 95%-interval for y^* via the percentiles of $y^*(i)$ to get:

$$p(1.46 \leq y^* \leq 3.64|\mathbf{y}, x^*) \approx 0.95. \quad (64)$$

End of solution



```

1 # keys
2 key1, key2 = random.split(key)
3
4 # generate samples
5 S = 1000
6 w_posterior = random.multivariate_normal(key1, w_MAP, Sigma, shape=(S))
7
8 # compute f(x*) for each posterior sample
9 fstar = jnp.array([model(3.75, wi) for wi in w_posterior])
10
11 # compute y* for each posterior sample
12 ystar = fstar + jnp.sqrt(sigma2)*random.normal(key2, shape=fstar.shape)
13
14 # compute statistics
15 ystar_mean = jnp.mean(ystar)
16 ystar_lower, ystar_upper = jnp.percentile(ystar, jnp.array([2.5, 97.5]), axis=-1)
17
18 # print
19 print(f'Mean of p(y|y, x*=3.75): {ystar_mean:.2f}')
20 print(f'95% interval for p(y|y, x*=3.75): [{ystar_lower:.2f}, {ystar_upper:.2f}]')
21

```

Figure 10: Contour plot of the posterior density and Laplace approximation